

Natural Language Processing

Session 4: **Tokenization and Stemming**

Instructor: Behrooz Mansouri
Spring 2023, University of Southern Maine

Previous Session

In the previous session, we learned about:

- ✓ Regular Expressions
- ✓ Word and Corpora
- ✓ Text Normalization
- ✓ String Edit Distance

Encoding

Text Representations: ASCII

Digital documents that are the input to an indexing process are typically bytes in a file; we need a sequence of characters

The first step of processing is to convert this byte sequence into a linear sequence of characters

Plain English text in **ASCII** (American Standard Code for Information Interchange) encoding; consists of a 7-bit code with 128 symbols (95 graphic symbols and 33 control symbols)

ASCII encoding:

0 1 0 0 0 0 1 = 65 (decimal)

0 1 0 0 0 0 1 = 101 (octal)

0 1 0 0 0 0 1 = 41 (hexadecimal)

Some codes are used as “control characters”

- e.g., 7 decimal, rings a “bell” (these days, a beep)

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	Space		64	40	100	@		96	60	140	#96;	
1	1	001	SOH (start of heading)	33	21	041	!		65	41	101	A		97	61	141	#97;	a
2	2	002	STX (start of text)	34	22	042	"		66	42	102	B		98	62	142	#98;	b
3	3	003	ETX (end of text)	35	23	043	#		67	43	103	C		99	63	143	#99;	c
4	4	004	EOT (end of transmission)	36	24	044	\$		68	44	104	D		100	64	144	#100;	d
5	5	005	ENQ (enquiry)	37	25	045	%		69	45	105	E		101	65	145	#101;	e
6	6	006	ACK (acknowledge)	38	26	046	&		70	46	106	F		102	66	146	#102;	f
7	7	007	BEL (bell)	39	27	047	'		71	47	107	G		103	67	147	#103;	g
8	8	010	BS (backspace)	40	28	050	(		72	48	110	H		104	68	150	#104;	h
9	9	011	TAB (horizontal tab)	41	29	051	)		73	49	111	I		105	69	151	#105;	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*		74	4A	112	J		106	6A	152	#106;	j
11	B	013	VT (vertical tab)	43	2B	053	+		75	4B	113	K		107	6B	153	#107;	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,		76	4C	114	L		108	6C	154	#108;	l
13	D	015	CR (carriage return)	45	2D	055	-		77	4D	115	M		109	6D	155	#109;	m
14	E	016	SO (shift out)	46	2E	056	.		78	4E	116	N		110	6E	156	#110;	n
15	F	017	SI (shift in)	47	2F	057	/		79	4F	117	O		111	6F	157	#111;	o
16	10	020	DLE (data link escape)	48	30	060	0		80	50	120	P		112	70	160	#112;	p
17	11	021	DC1 (device control 1)	49	31	061	1		81	51	121	Q		113	71	161	#113;	q
18	12	022	DC2 (device control 2)	50	32	062	2		82	52	122	R		114	72	162	#114;	r
19	13	023	DC3 (device control 3)	51	33	063	3		83	53	123	S		115	73	163	#115;	s
20	14	024	DC4 (device control 4)	52	34	064	4		84	54	124	T		116	74	164	#116;	t
21	15	025	NAK (negative acknowledge)	53	35	065	5		85	55	125	U		117	75	165	#117;	u
22	16	026	SYN (synchronous idle)	54	36	066	6		86	56	126	V		118	76	166	#118;	v
23	17	027	ETB (end of trans. block)	55	37	067	7		87	57	127	W		119	77	167	#119;	w
24	18	030	CAN (cancel)	56	38	070	8		88	58	130	X		120	78	170	#120;	x
25	19	031	EM (end of medium)	57	39	071	9		89	59	131	Y		121	79	171	#121;	y
26	1A	032	SUB (substitute)	58	3A	072	:		90	5A	132	Z		122	7A	172	#122;	z
27	1B	033	ESC (escape)	59	3B	073	;		91	5B	133	[		123	7B	173	#123;	{
28	1C	034	FS (file separator)	60	3C	074	<		92	5C	134	\		124	7C	174	#124;	
29	1D	035	GS (group separator)	61	3D	075	=		93	5D	135	^		125	7D	175	#125;	~
30	1E	036	RS (record separator)	62	3E	076	>		94	5E	136	_		126	7E	176	#126;	~
31	1F	037	US (unit separator)	63	3F	077	?		95	5F	137	~		127	7F	177	#127;	DEL

Text Representations: East Asian Character Sets

East Asian Character Sets: More than 256 characters are needed

- Two-byte encoding schemes (e.g., Extended Unix Code (EUC)) are used
- Several countries have unique character sets:
 - GB (Guobiao): China
 - BIG5: Taiwan
 - JIS(Japanese Industrial Standards): Japan
 - KS: Korea
 - TCVN: Vietnam
- Many characters appear in several languages
- Research Libraries Group developed EACC (East Asian Character Code) as a Unified “CJK” (Chinese, Japanese, and Korean) character set

Documents Representations: Unicode

Single code for “all” the world’s characters

- The Unicode is a standard for consistent encoding maintained by the [Unicode Consortium](#)
- Unicode can be implemented by different character sets. The most commonly used encodings are UTF-8 and UTF-16
- **UTF-8** encoding is designed for disk file systems
 - UTF-8 encoding is a variable-sized encoding scheme
 - Variable sized encoding means the code points are represented using 1, 2, 3 or 4 bytes depending on their size
 - UTF-8 is the preferred encoding for email and web pages
- **UTF-16** encoding is a variable-length (2 or 4 bytes) character encoding for Unicode
 - UTF-16 is the only web-encoding incompatible with **ASCII** and never gained popularity on the web
 - UTF-16 is used in major operating systems and environments, like Microsoft Windows, Java and .NET

Tokenization

What is Tokenization?

Tokenization is the task of chopping text into pieces, called tokens

Token is an instance of a sequence of characters in some particular document (grouped as a useful semantic unit)

Type is the class of all tokens containing the same character sequence

Term is a (perhaps normalized) type that is included in the corpus dictionary

Example: to sleep more to learn

Token: to, sleep, more, to, learn

Type: to, sleep, more, learn

Term: sleep, more, learn (stop words removed)

Tokenization Issues

- **Maine's state capital**
 - Maine, Maines, Maine's?
- **Dover-Foxcroft**
 - Dover-Foxcroft or two tokens: Dover, Foxcroft
 - State-of-the-art: break up hyphenated sequence
- **San Francisco:** one token or two?
- **Handling numbers**
 - September 13, 2022
 - 13/9/2022
 - (686)753-2910
 - Older IR systems may not index numbers
- Will often index “meta-data” separately



Dover-Foxcroft, Maine

Tokenization Issues (Other Languages)

- **French**

- L'ensemble → one token or two?
- L ? L' ? Le ?
- Want l'ensemble to match with un ensemble

- **German**

- Noun compound are not segmented
- Kraftfahrzeughaftpflichtversicherung
- 'motor vehicle indemnity insurance'

- **Chinese & Japanese**

- No space between words
- 自然语言处理课程 → 'Natural Language Processing course'

- **Persian & Arabic**

- Right to left
- Letters can be connected together
- کلاس پردازش زبان طبیعی = کلاس پردازش زبان طبیعی
- 'Natural language processing class'

Word-based Tokenization

Approach

- Splitting the text by spaces
- Other delimiters such as punctuation can be used

Advantages

- Easy to implement

Disadvantages

- High risk of missing words; e.g., Let and Let's will have two different types
- Languages like Chinese do not have space
- Huge vocabulary size (token type)
 - Limit the number of words that can be added to the vocabulary
- Misspelled words will be considered as a token

Character-based Tokenization

Approach

- Splitting the text into individual characters

Advantages

- There will be no or very few unknown words (Out Of Vocabulary)
- Useful for languages that characters carry information
- Fewer number of tokens
- Easy to implement

what is glablahglee?

I'm sorry, but I couldn't find any information about "glablahglee". Could you provide more context or clarify the term you're asking about?

Disadvantages

- A character usually does not have a meaning
 - Cannot learn semantic for words
- Larger sequence to be processed by models
 - More input to process

Subword Tokenization

Approach

- Frequently used words should not be split into smaller subwords
- Rare words should be decomposed into meaningful subwords
- Uses a special symbol to indicate which word is the start of the token and which word is the completion of the start of the token
 - Tokenization → “Token”, “##ization”
- State-of-the-art approaches for NLP and IR rely on this type

Subword Tokenization

Approach

- Frequently used words should not be split into smaller subwords
- Rare words should be decomposed into meaningful subwords
- Uses a special symbol to indicate which word is the start of the token and which word is the completion of the start of the token
 - Tokenization → “Token”, “##ization”
- State-of-the-art approaches for NLP and IR rely on this type

Advantages

- Out-of-vocabulary word problem solved
- Manageable vocabulary sizes

Disadvantages

- New scheme and needs more exploration

Byte Per Encoding (BPE) and WordPiece are two examples of this scheme

Byte-Pair Encoding (BPE) Tokenization

Uses Huffman encoding for tokenization (greedy algorithm)

Training Steps:

1. Starts with splitting the input words into single characters
(each of them corresponds to a symbol in the final vocabulary)
* In practice we commonly add special end of word symbol “__” before space
2. Find the most frequent occurring pair of symbols from the current vocabulary
3. Add this to the vocabulary and size of vocabulary increases by one
4. Repeat steps (2) and (3) till the defined number of tokens are built
or no new combination of symbols exist with required frequency

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

Corpus

5 l o w ____
2 l o w e s t ____
6 n e w e r ____
3 w i d e r ____
2 n e w ____

Vocabulary

____, d, e, i, l, n, o, r, s, t, w



Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

9 times

	Corpus
5	l o w _ _
2	l o w e s t _ _
6	n e w e r _ _
3	w i d e r _ _
2	n e w _ _

Vocabulary

_, d, e, i, l, n, o, r, s, t, w

Vocabulary

_, d, e, i, l, n, o, r, s, t, w, **er**

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

	Corpus	Vocabulary
9 times	5 low __	__, d, e, i, l, n, o, r, s, t, w, er
	2 low est __	
	6 new er __	Vocabulary
	3 wider er __	__, d, e, i, l, n, o, r, s, t, w, er, er__
	2 new __	

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

	Corpus	Vocabulary
8 times	5 low__	__, d, e, i, l, n, o, r, s, t, w, er, er__
	2 lowest__	
	6 new__	Vocabulary
	3 wider__	__, d, e, i, l, n, o, r, s, t, w, er, er__, ne
	2 new__	

Byte-Pair Encoding (BPE) Tokenization

Training corpus: low low low low low lowest lowest newer newer newer newer
newer newer wider wider wider new new

8 times

	Corpus
5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

Vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

Vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne, **new**

Byte-Pair Encoding (BPE) Tokenization

Corpus

5 l o w __
2 l o w e s t __
6 n e w e r __
3 w i d e r __
2 n e w __

Final Vocabulary

__, d, e, i, l, n, o, r, s, t, w, er, er__, ne, new, lo, low, newer __, low__

Using BPE for tokenization:

Input: newer__ → **Tokens: newer__**

Input: lower__ → **Tokens: low, er__**

Merge based on the order we learned:

er → er__ → ne → new → newer__

er → er__ → lo → low

WordPiece Tokenization

Assignment!

How WordPiece is different from BPE?

Use the example corpus from the previous slide to show how WordPiece tokenization work

HuggingFace: <https://huggingface.co/course/chapter6/6?fw=pt>

Paper: <https://arxiv.org/pdf/1609.08144v2.pdf>

Google Blog: <https://ai.googleblog.com/2021/12/a-fast-wordpiece-tokenization-system.html>

Stop Words

Stopwords Removal

Stopping: Removing common words from the stream of tokens that become index terms

- Words that are function words helping form sentence structure: the, of, and, to,
- For an application, an additional domain specific stop words list may be constructed
- Why do we need to remove stop words?
 - Reduce indexing (or data) file size
 - Usually has no impact on the NLP task's effectiveness, and may even improve it

Stopwords Removal

Stopping: Removing common words from the stream of tokens that become index terms

- Words that are function words helping form sentence structure: the, of, and, to,
- For an application, an additional domain specific stop words list may be constructed
- Why do we need to remove stop words?
 - Reduce indexing (or data) file size
 - Usually has no impact on the NLP task's effectiveness, and may even improve it
- Can sometimes cause issues for NLP tasks:
 - e.g., phrases: "to be or not to be", "let it be", "flights to Portland Maine"
 - Some tasks consider very small stopwords list
 - Sometimes perhaps only "the"
- List of stopwords: <https://www.ranks.nl/stopwords>

Normalization

Token Normalization

Token/Term Normalization: Reducing tokens to canonical token (creating equivalence classes)

- Deleting periods to form a term
 - U.S.A, USA
 - How about C.A.T (Caterpillar Inc) and CAT? (Test Google on this!)
- Deleting hyphens to form a term
 - anti-discriminatory, antidiscriminatory
- Keeping relationships between unnormalized tokens
 - car, automobile
- British vs. American spelling:
 - color vs. colour
- Other languages:
 - French: résumé vs. resume

Stemming

Stemming

Stemming: To group words that are derived from a common stem

- e.g, “fish”, “fishes”, “fishing” could be mapped to “fish”
- Generally produces small improvements in tasks effectiveness
- Similar to stopping, stemming can be done aggressively, conservatively, or not at all
 - Aggressively: consider “fish” and “fishing” the same
 - Conservatively: just identifying plural forms using the letter “s”
 - issues: ‘Centuries’ → ‘Centurie’
 - Not at all: Consider all the word variants

Stemming

Stemming: To group words that are derived from a common stem

- e.g, “fish”, “fishes”, “fishing” could be mapped to “fish”
- Generally produces small improvements in tasks effectiveness
- Similar to stopping, stemming can be done aggressively, conservatively, or not at all
 - Aggressively: consider “fish” and “fishing” the same
 - Conservatively: just identifying plural forms using the letter “s”
 - issues: ‘Centuries’ → ‘Centurie’
 - Not at all: Consider all the word variants
- In different languages, stemming can have different importance for effectiveness:
 - In Arabic, morphology is more complicated than English
 - In Chinese, stemming is not effective

Evaluation of Stemmers

There are three criteria for evaluating stemmers:

1. Correctness
2. Efficiency of the task
3. Compression performance

There are two ways in which stemming can be incorrect:

- Over-stemming (too much of the term is removed)
 - Two or more words being reduced to the same wrong root
 - e.g., 'centennial', 'century', 'center': 'cent'
- Under-stemming (too little of the term is removed)
 - Two or more words could be wrongly reduced to more than one root word
 - e.g., 'acquire', 'acquiring', 'acquired': 'acquir' 'acquisition': 'acquis'

Porter Stemmer (1980)

The most common stemmer for English, introduced by Martin Porter

A rule-based stemmer with rules for mostly suffix-stripping such as:

- “ing” → “-” connecting → connect
- “sses” → “ss” caresses → caress
- (m>0) “EED” → “EE” feed → feed agreed → agree
 - m=measure of word or word part, when represented in form (VC⁺)
 - V= vowel and C=consonants



Porter Stemmer (1980)

The most common stemmer for English, introduced by Martin Porter

A rule-based stemmer with rules for mostly suffix-stripping such as:

- “ing” → “-” connecting → connect
- “sses” → “ss” caresses → caress
- (m>0) “EED” → “EE” feed → feed agreed → agree
 - m=measure of word or word part, when represented in form (VC)
 - V= vowel and C=consonants



Advantage: It produces the best output as compared to other stemmers, and it has less error rate

Disadvantage: Morphological variants produced are not always real words (produces stems)

Krovetz Stemmer (1993)

Hybrid algorithmic-dictionary-based method introduced by Robert Krovetz

Word checked in dictionary:

- If present, either left alone (or replaced with “exception”)
- If not present, word is checked for suffixes that could be removed
- After removal, dictionary is checked again



Advantage: Produces words not stems

Disadvantage: Lower false positive rate, somewhat higher false negative

FP: Policy → Police FN: noise/noisy

Example

Original text:

Document will describe marketing strategies carried out by U.S. companies for their agricultural chemicals, report predictions for market share of such chemicals, or report market statistics for agrochemicals, pesticide, herbicide, fungicide, insecticide, fertilizer, predicted sales, market share, stimulate demand, price cut, volume of sales.

Porter stemmer:

document describ market strategi carri compani agricultur chemic report predict market share chemic
report market statist agrochem pesticid herbicid fungicid insecticid fertil predict sale market share
stimul demand price cut volum sale

Krovetz stemmer:

document describe marketing strategy carry company agriculture chemical report prediction market
share chemical report market statistic agrochemic pesticide herbicide fungicide insecticide fertilizer
predict sale stimulate demand price cut volume sale

Lemmatization

Lemmatization

Reduce inflectional/variant forms to base form

- am, are, is → be
- car, cars, car's, cars' → car
- the boy's cars are different colors → the boy car be different color

Lemmatization implies doing “proper” reduction to **dictionary** headword form

- e.g., WordNet is a lexical database of semantic relations between words in more than 200 languages

Stemming is a process that extract stems by removing last few characters from a word, often leading to incorrect meanings and spelling

Lemmatization considers the context and converts the word to its meaningful base form, which is called Lemma

Phrases

Phrases

In a task such as information retrieval, input queries can be 2-3 word phrases

- Phrases can yield more precise queries
 - “University of Southern Maine”, “black sea”
- Less ambiguous
 - “Red apple” vs. “apple”

Phrase is any sequence of n words: n -gram

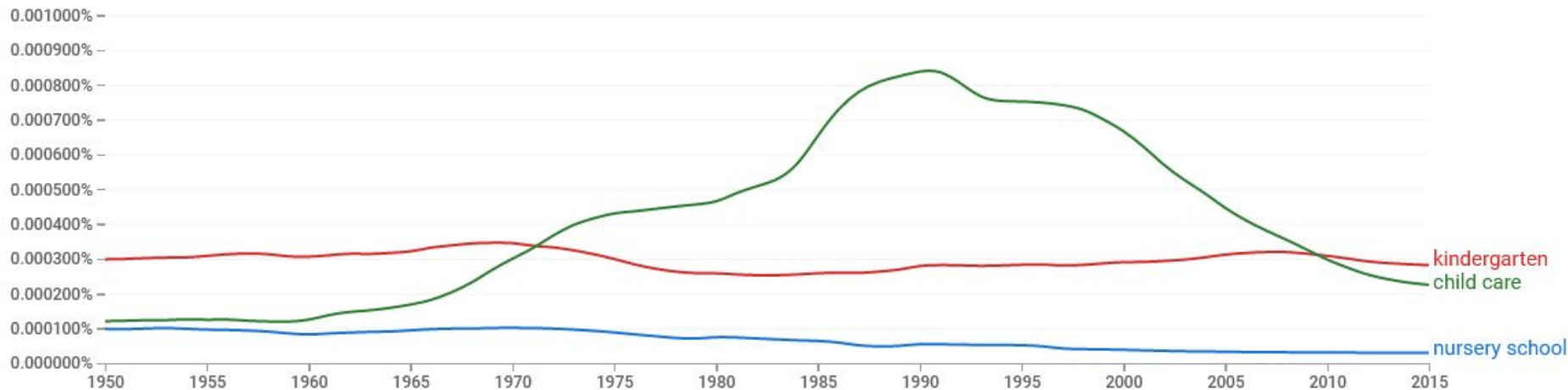
- **unigram**: one word **bigram**: sequence of 2 words **trigram**: sequence of 3 words
- Generated by:
 - Choosing a particular value for ‘ n ’
 - Moving that “window” forward one unit word at a time
- The more frequently a word n -gram occurs, the more likely it is to correspond to a meaningful phrase in the language

Google N-gram

Number of tokens:	1,024,908,267,229
Number of sentences:	95,119,665,584
Number of unigrams:	13,588,391
Number of bigrams:	314,843,401
Number of trigrams:	977,069,902
Number of fourgrams:	1,313,818,354
Number of fivegrams:	1,176,470,663

<https://ai.googleblog.com/2006/08/all-our-n-gram-are-belong-to-you.html>

Google N-gram



Y-axis: of all the bigrams contained in our sample of books written in English and published in the United States, what percentage of them are "nursery school" or "child care"

<https://books.google.com/ngrams/info#>
<https://books.google.com/ngrams/>

Zipf's Law

Distribution of word frequencies is very skewed

- Few words occur very often, many hardly ever occur
- Two most common words (“the”, “of”) make up about 10% of all word occurrences in text documents

The frequency of r^{th} most common word is inversely proportional to r

- the rank of a word times its frequency (f) is approximately a constant (k)
- $r \times f \approx k \rightarrow r \times P_r \approx c$ (P_r is probability of word occurrence)



George Kingsley Zipf

N-grams form a Zipf distribution

Zipf's Law



Rank versus probability of occurrence for words assuming Zipf's law ($\text{rank} \times \text{probability} = 0.1$)

NLTK: Natural Language Toolkit

<https://www.nltk.org/>

Tokenization & Stopword Removal

```
import nltk
```

```
nltk.download('punkt')  
text = "I love natural language processing course at the University of Southern Maine!"  
token_words = nltk.word_tokenize(text)  
print(token_words)
```

```
['I', 'love', 'natural', 'language', 'processing', 'course', 'at', 'the', 'University', 'of', 'Southern', 'Maine', '!']
```

```
import nltk  
nltk.download('stopwords')  
from nltk.corpus import stopwords  
from nltk.tokenize import word_tokenize  
text = "I love natural language processing course at the University of Southern Maine!"  
stop_words = set(stopwords.words('english'))  
word_tokens = word_tokenize(text)  
filtered_text = [w for w in word_tokens if not w.lower() in stop_words]  
print(word_tokens)  
print(filtered_text)
```

```
['I', 'love', 'natural', 'language', 'processing', 'course', 'at', 'the', 'University', 'of', 'Southern', 'Maine', '!']  
['love', 'natural', 'language', 'processing', 'course', 'University', 'Southern', 'Maine', '!']
```

Stemming & Lemmatization

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

ps = PorterStemmer()
text = "I love natural language processing course at the University of Southern Maine!"
words = word_tokenize(text)
for w in words:
    print(w, ":", ps.stem(w))
```

```
I : i
love : love
natural : natur
language : languag
processing : process
course : cours
at : at
the : the
University : univers
of : of
Southern : southern
Maine : main
! : !
```

```
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
nltk.download('wordnet')
nltk.download('omw-1.4')
lemmatizer = WordNetLemmatizer()
text = "I love natural language processing course at the University of Southern Maine!"
words = word_tokenize(text)
for w in words:
    print(w, ":", lemmatizer.lemmatize(w))
```

```
I : I
love : love
natural : natural
language : language
processing : processing
course : course
at : at
the : the
University : University
of : of
Southern : Southern
Maine : Maine
! : !
```

WE NEED TO LOOK CAREFULLY!

- You should always make sure to look at the data and see what are the tokens after the tokenization → What is being indexed?
- Use small input small and manually observe the output
- In many researches, the default parameters have handled the input differently than expected
 - Using HTML files, how the tags are handled? What encoding is used?
 - Check what kind of normalization is applied.
 - Decided whether the stop-words are removed? (or should be removed)

WHAT HAVE
YOU LEARNED?

Summary

Today we learned about:



How documents are encoded



Processing text with:

- Tokenization
- Stopword removal
- Normalization
- Stemming
- Lemmetization

Next Session

N-Gram and Language Models

We will explore

- Language Models and Evaluation
- Markov Assumption
- Estimating N-grams and Smoothing

To do:

- Submit Assignment 1
- **Reading:** Chapter [2](#) and [3](#) of Jurafsky Book

Words and Transducers

Morphological Analysis

Morphology is the study of the internal structure of words

Words structure is analyzed by composition of **morphemes** - the smallest units for grammatical analysis:

- Boys: boy-s
- Friendlier: friend-ly-er
- Ungrammaticality: un-grammat-ic-al-ity

Semitic languages, like Hebrew and Arabic, are based on templates and roots

We will concentrate on affixation-based languages, in which words are composed of stems and affixes

Stems and Affixes

It is often useful to distinguish two broad classes of morphemes:

- Stems
 - The **stem** is the “main” morpheme of the word, supplying the main meaning
 - example; In Cat’s, Cat is stem
- Affixes
 - The **affixes** add “additional” meanings of various kinds
 - example; In Cat’s, ’s is affixes

Categories of Affixes

Affixes are further divided into 4 types:

- **Prefixes** precede the stem
 - e.g., The word unbuckle is composed of a stem buckle and the prefix un-
- **Suffixes** follow the stem
 - e.g., the word eats is composed of a stem eat and the suffix -s
- **Infixes**, are inserted inside the stem; a morpheme is inserted in the middle of a word
 - e.g., the affix e, infixed to the stem bled “borrow” to produce bleed
 - the affix um, infixed to the stem hingi “borrow” to produce humingi.
- **Circumfixes**, do both (prefixes and suffixes).
 - English doesn't have any good examples of circumfixes, but many other languages do
 - In German, e.g., adding ge- to the beginning of the stem and -t to the end; so the past participle of the verb sagen (to say) is gesagt (said).

Morphology to Create Words

There are many ways to combine morphemes to create words

Four methods are common and play important roles in speech and language processing:

- (1) Inflection,
- (2) Derivation,
- (3) Cliticization
- (4) Compounding

Inflection

It is the combination of a word stem with a grammatical morpheme (in-category; paradigmatic)

Usually resulting in a word of the same class as the original stem, and usually filling some syntactic function like agreement

- English has the inflectional morpheme -s for marking the plural on nouns
- The inflectional morpheme -ed for marking the past tense on verbs

For example:

Play > Played

Player > Players

Inflection: Regular vs Irregular

Morphological Form Classes	Regularly Inflected Verbs		
stem	walk	merge	try
-s form	walks	merges	tries
-ing participle	walking	merging	trying
Past form or -ed participle	walked	merged	tried

Morphological Form Classes	Irregularly Inflected Verbs		
stem	eat	catch	cut
-s form	eats	catches	cuts
-ing participle	eating	catching	cutting
Past form or -ed participle	ate/eaten	caught	cut

Derivation

Is the combination of a word stem with a grammatical morpheme
(between-categories; non-paradigmatic)

- mainly deal with adjective, nouns and verbs

Resulting in a word of a different class, often with a meaning hard to predict exactly

- the verb computerize can take the derivational suffix **-ation** to produce the noun computeriz**ation**

Cliticization

It is the combination of a word stem with a clitic

A clitic is a morpheme that acts syntactically like a word, but is reduced in form and attached (phonologically and sometimes orthographically) to another word

- English morpheme 've in the word “ I've ” is a clitic

Full Form	Clitic	Full Form	Clitic
am	'm	have	've
are	're	has	's
is	's	had	'd
will	'll	would	'd

Compounding

It is the combination of multiple word stems together

- The noun doghouse is the concatenation of the morpheme dog with the morpheme house
- Baseball → base + ball
- Desktop → desk + top

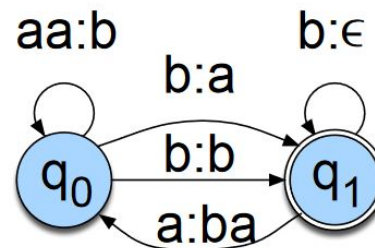
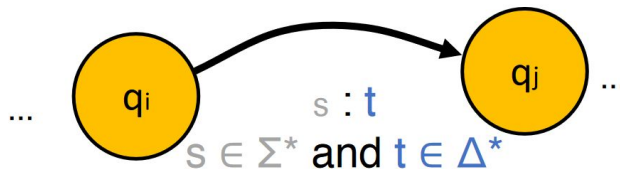
Finite-state Morphological Parsing

A transducer maps between one representation and another

A finite-state transducer or FST is a type of finite automaton which maps between two sets of symbols

Another way of looking at an FST is as a machine that reads one string and generates another

- Q : a finite set of states
- $q_0 \in Q$: a special start state
- $F \subseteq Q$: a set of final states
- Σ and Δ : two finite alphabets
- Transitions:



A finite-state transducer,
modified from Mohri (1997)

Finite-state Morphological Parsing: Example

Generating Inflected forms of English Verbs from Lemmas

